

RAG infrastructure for generative AI using Vertex AI and Vector Search



Last reviewed 2025-03-07 UTC

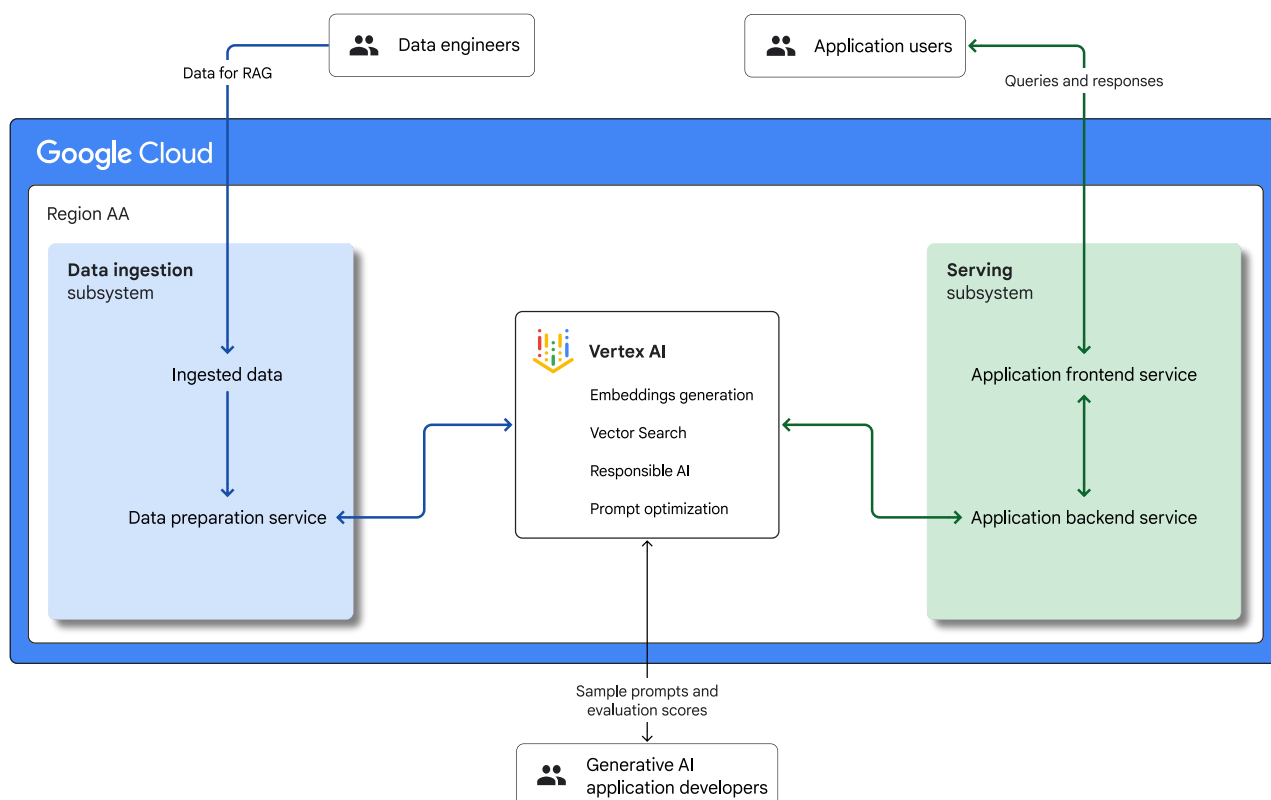
This document provides a reference architecture that you can use to design the infrastructure for a generative AI application with retrieval-augmented generation (RAG) (https://developers.google.com/machine-learning/glossary#retrieval-augmented_generation) by using Vector Search (</vertex-ai/docs/vector-search/overview>). Vector Search is a fully managed Google Cloud service that provides optimized serving infrastructure for very large-scale vector-similarity matching.

The intended audience for this document includes architects, developers, and administrators of generative AI applications. The document assumes a basic understanding of AI, machine learning (ML), and large language model (LLM) concepts. This document doesn't provide guidance about how to design and develop a generative AI application.

Note: Depending on your requirements for application hosting and AI infrastructure, you can consider other architectural approaches for your RAG-capable generative AI application. For more information, see the "Design alternatives" section later in this document.

Architecture

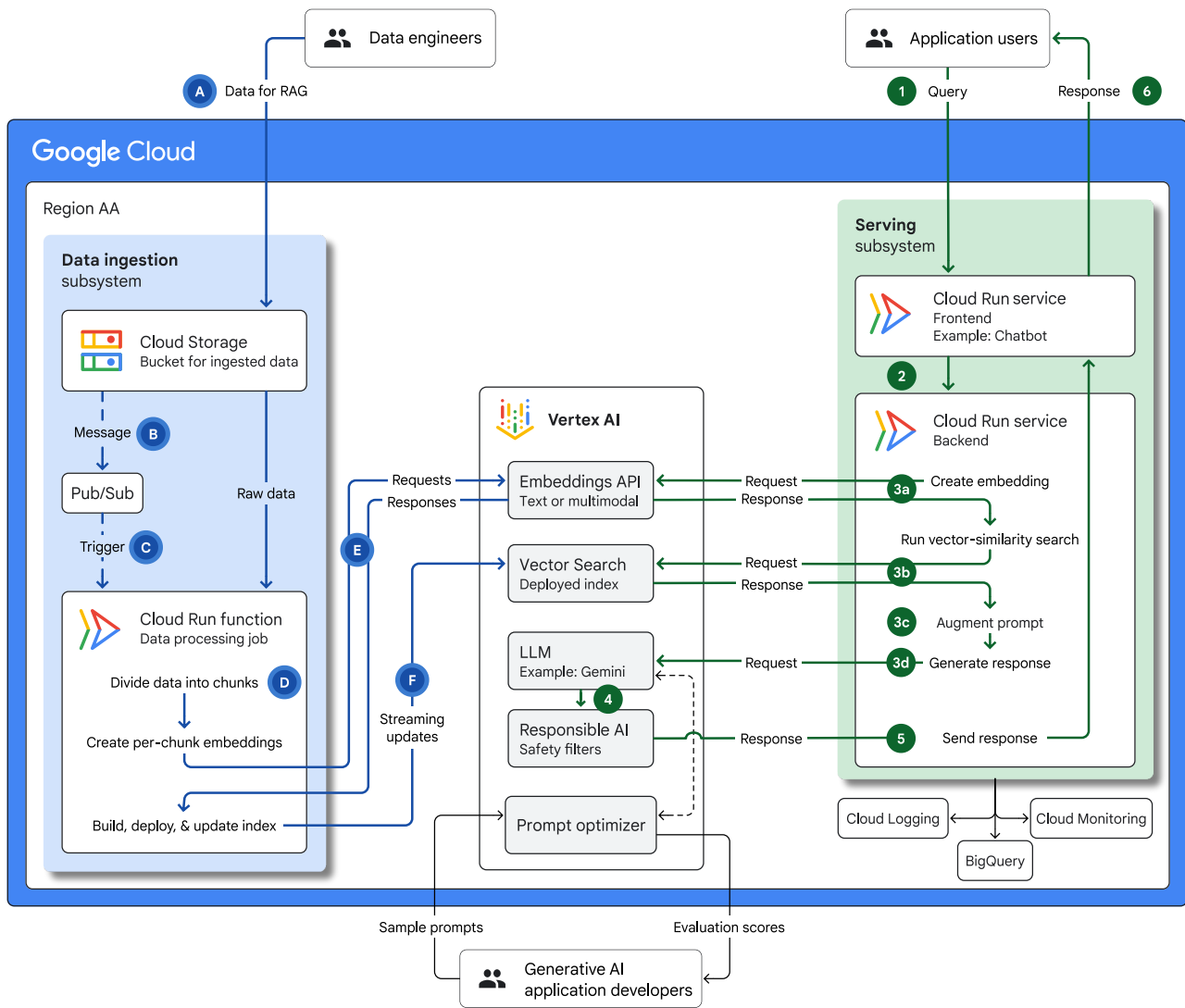
The following diagram shows a high-level view of the architecture that this document presents:



The architecture in the preceding diagram has two subsystems: data ingestion and serving.

- The **data ingestion subsystem** ingests data that's uploaded from external sources. The subsystem prepares the data for RAG and interacts with Vertex AI to generate [embeddings](/vertex-ai/docs/glossary#embedding) for the ingested data and to build and update the [vector index](/vertex-ai/docs/vector-search/overview#terminology).
- The **serving subsystem** contains the generative AI application's frontend and backend services.
 - The frontend service handles the query-response flow with application users and forwards queries to the backend service.
 - The backend service uses Vertex AI to generate query embeddings, perform vector-similarity search, and apply [Responsible AI](/vertex-ai/generative-ai/docs/learn/responsible-ai) safety filters and system instructions.

The following diagram shows a detailed view of the architecture:



The following sections describe the data flow within each subsystem of the preceding architecture diagram.

Data ingestion subsystem

The data ingestion subsystem ingests data from external sources and prepares the data for RAG. The following are the steps in the data-ingestion and preparation flow:

- A. Data is uploaded from external sources to a Cloud Storage bucket (/storage/docs/buckets). The external sources might be applications, databases, or streaming services.
- B. When data is uploaded to Cloud Storage, a message is published to a Pub/Sub topic (/pubsub/docs/publish-message-overview).
- C. When the Pub/Sub topic receives a message, it triggers a Cloud Run function (/functions/docs/concepts/overview).

- D. The Cloud Run function parses the raw data, formats it as required, and divides it into chunks.
- E. The function uses the [Vertex AI Embeddings API](#) (/vertex-ai/generative-ai/docs/embeddings) to create embeddings of the chunks by using an embedding model that you specify. Vertex AI supports [text](#) (/vertex-ai/generative-ai/docs/embeddings/get-text-embeddings#supported-models) and [multimodal](#) (/vertex-ai/generative-ai/docs/embeddings/get-multimodal-embeddings#supported-models) embedding models.
- F. The function then builds a Vector Search index of the embeddings and then deploys the index.

When new data is ingested, the preceding steps are performed for the new data and the index is updated using [streaming updates](#) (/vertex-ai/docs/vector-search/update-rebuild-index#upsert-streaming).

When the serving subsystem processes user requests, it uses the Vector Search index for vector-similarity search. The next section describes the serving flow.

Serving subsystem

The serving subsystem handles the query-response flow between the generative AI application and its users. The following are the steps in the serving flow:

1. A user submits a natural-language query to a [Cloud Run service](#) (/run/docs/resource-model#services) that provides a frontend interface (such as a chatbot) for the generative AI application.
2. The frontend service forwards the user query to a backend Cloud Run service.
3. The backend service processes the query by doing the following:
 - a. Converts the query to embeddings by using the same embeddings model and parameters that the data ingestion subsystem uses to generate embeddings of the ingested data.
 - b. Retrieves relevant [grounding data](#) (/vertex-ai/docs/glossary#ground_truth) by performing a vector-similarity search for the query embeddings in the Vector Search index.
 - c. Constructs an augmented prompt by combining the original query with the grounding data.

- d. Sends the augmented prompt to an LLM that's deployed on Vertex AI.
4. The LLM generates a response.
5. For each prompt, Vertex AI applies the Responsible AI safety filters that you've configured and then sends the filtered response and AI safety scores to the Cloud Run backend service.
6. The application sends the response to the user through the Cloud Run frontend service.

You can store and view logs of the query-response activity in Cloud Logging, and you can set up logs-based monitoring by using Cloud Monitoring. You can also load the generated responses into BigQuery for offline analytics.

The [Vertex AI prompt optimizer](/vertex-ai/generative-ai/docs/learn/prompts/prompt-optimizer) (/vertex-ai/generative-ai/docs/learn/prompts/prompt-optimizer) helps you improve prompts at scale, both during initial prompt design and for ongoing prompt tuning. The prompt optimizer evaluates your model's response to a set of sample prompts that ML engineers provide. The output of the evaluation includes the model's responses to the sample prompts, scores for metrics that the ML engineers specify, and a set of optimized system instructions that you can consider using.

Products used

This reference architecture uses the following Google Cloud products:

- [Vertex AI](https://cloud.google.com/vertex-ai) (https://cloud.google.com/vertex-ai): An ML platform that lets you train and deploy ML models and AI applications, and customize LLMs for use in AI-powered applications.
- [Vector Search](/vertex-ai/docs/vector-search/overview) (/vertex-ai/docs/vector-search/overview): A vector similarity-matching service that lets you store, index, and search semantically similar or related data.
- [Cloud Run](https://cloud.google.com/run) (https://cloud.google.com/run): A serverless compute platform that lets you run containers directly on top of Google's scalable infrastructure.
- [Cloud Run functions](https://cloud.google.com/functions) (https://cloud.google.com/functions): A serverless compute platform that lets you run single-purpose functions directly in Google Cloud.
- [Cloud Storage](https://cloud.google.com/storage) (https://cloud.google.com/storage): A low-cost, no-limit object store for diverse data types. Data can be accessed from within and outside Google Cloud, and it's replicated across locations for redundancy.

- Pub/Sub (<https://cloud.google.com/pubsub>): An asynchronous and scalable messaging service that decouples services that produce messages from services that process those messages.
- Cloud Logging (<https://cloud.google.com/logging>): A real-time log management system with storage, search, analysis, and alerting.
- Cloud Monitoring (<https://cloud.google.com/monitoring>): A service that provides visibility into the performance, availability, and health of your applications and infrastructure.
- BigQuery (<https://cloud.google.com/bigquery>): An enterprise data warehouse that helps you manage and analyze your data with built-in features like machine learning geospatial analysis, and business intelligence.

Use cases

RAG is an effective technique to improve the quality of output that's generated from an LLM. This section provides examples of use cases for which you can use RAG-capable generative AI applications.

Personalized product recommendations

An online shopping site might use an LLM-powered chatbot to assist customers with finding products or getting shopping-related help. The questions from a user can be augmented by using historical data about the user's buying behavior and website interaction patterns. The data might include user reviews and feedback that's stored in an unstructured datastore or search-related metrics that are stored in a web analytics data warehouse. The augmented question can then be processed by the LLM to generate personalized responses that the user might find more appealing and compelling.

Clinical assistance systems

Doctors in hospitals need to quickly analyze and diagnose a patient's health condition to make decisions about appropriate care and medication. A generative AI application that uses a medical LLM like Med-PaLM (<https://sites.research.google/med-palm/>) can be used to assist doctors in their clinical diagnosis process. The responses that the application generates can be grounded in historical patient records by contextualizing the doctors' prompts with data from the hospital's electronic health record (EHR) database or from an external knowledge base like PubMed (<https://pubmed.ncbi.nlm.nih.gov/>).

Efficient legal research

Generative AI-powered legal research lets lawyers quickly query large volumes of statutes and case laws to identify relevant legal precedents or summarize complex legal concepts. The output of such research can be enhanced by augmenting a lawyer's prompts with data that's retrieved from the law firm's proprietary corpus of contracts, past legal communication, and internal case records. This design approach ensures that the generated responses are relevant to the legal domain that the lawyer specializes in.

Design alternatives

This section presents alternative design approaches that you can consider for your RAG-capable generative AI application in Google Cloud.

AI infrastructure alternatives

If you want to take advantage of the vector store capabilities of a fully managed Google Cloud database like AlloyDB for PostgreSQL or Cloud SQL for your RAG application, then see [RAG infrastructure for generative AI using Vertex AI and AlloyDB for PostgreSQL](#) (/architecture/rag-capable-gen-ai-app-using-vertex-ai).

If you want to rapidly build and deploy RAG-capable generative AI applications by using open source tools and models Ray, Hugging Face, and LangChain, see [RAG infrastructure for generative AI using GKE and Cloud SQL](#) (/architecture/rag-capable-gen-ai-app-using-gke).

Application hosting options

In the architecture that's shown in this document, Cloud Run is the host for the generative AI application and data processing. Cloud Run is a developer-focused, fully managed application platform. If you need greater configuration flexibility and control over the compute infrastructure, you can deploy your application to [GKE](#) (/kubernetes-engine/docs) clusters or to [Compute Engine](#) (/compute) VMs.

The decision of whether to use Cloud Run, GKE, or Compute Engine as your application host involves trade-offs between configuration flexibility and management effort. With the serverless Cloud Run option, you deploy your application to a preconfigured environment that requires minimal management effort. With Compute Engine VMs and GKE containers, you're responsible for managing the underlying compute resources, but you have greater

configuration flexibility and control. For more information about choosing an appropriate application hosting service, see the following documents:

- [Is my app a good fit for Cloud Run?](/run/docs/fit-for-run) (/run/docs/fit-for-run)
- [Select a managed container runtime environment](/architecture/select-managed-container-runtime-environment) (/architecture/select-managed-container-runtime-environment)
- [Hosting Applications on Google Cloud](https://cloud.google.com/hosting-options) (https://cloud.google.com/hosting-options)

Other options

For information about other infrastructure options, supported models, and grounding techniques that you can use for generative AI applications in Google Cloud, see [Choose models and infrastructure for your generative AI application](/docs/generative-ai/choose-models-infra-for-ai) (/docs/generative-ai/choose-models-infra-for-ai).

Design considerations

This section describes design factors, best practices, and design recommendations that you should consider when you use this reference architecture to develop a topology that meets your specific requirements for security, reliability, cost, and performance.

The guidance in this section isn't exhaustive. Depending on the specific requirements of your application and the Google Cloud and third-party products and features that you use, there might be additional design factors and trade-offs that you should consider.

Security, compliance, and privacy

This section describes design considerations and recommendations to design a topology in Google Cloud that meets the security and compliance requirements of your workloads.

Product	Design considerations and recommendations
Vertex AI	Security controls: Vertex AI supports Google Cloud security controls that you can use to meet your requirements for data residency, data encryption, network security, and access transparency. For more information, see Security controls for Vertex AI (/vertex-ai/docs/general/vertexai-security-controls) and Security controls for Generative AI (/vertex-ai/docs/generative-ai/genai-security-controls). Model access: You can set up organization policies to limit the type and versions of LLMs that can be used in a Google Cloud project. For more information, see Control access to

Product	Design considerations and recommendations
	<u>Model Garden models</u> (/vertex-ai/generative-ai/docs/control-model-access). Shared responsibility: Vertex AI secures the underlying infrastructure and provides tools and security controls to help you protect your data, code, and models. For more information, see <u>Vertex AI shared responsibility</u> (/vertex-ai/docs/shared-responsibility). Data protection: Use the <u>Cloud Data Loss Prevention API</u> (/sensitive-data-protection/docs/apis) to discover and de-identify sensitive data, such as personally identifiable information (PII), in the prompts and responses and in log data. For more information, see this video: <u>Protecting sensitive data in AI apps</u> (https://www.youtube.com/watch?v=ct1nuLS-wIA).
Cloud Run	Ingress security (frontend service): To control external access to the application, <u>disable the default run.app URL</u> (/run/docs/securing/ingress#disable-url) of the frontend Cloud Run service and <u>set up a regional external Application Load Balancer</u> (/load-balancing/docs/https/setting-up-reg-ext-https-serverless). Along with load-balancing incoming traffic to the application, the load balancer handles SSL certificate management. For added protection, you can use <u>Google Cloud Armor security policies</u> (/armor/docs/integrating-cloud-armor#serverless) to provide request filtering, DDoS protection, and rate limiting for the service. Ingress security (backend service): The Cloud Run service for the application's backend in this architecture doesn't need access from the internet. To ensure that only internal clients can access the service, set the <code>ingress</code> parameter to <code>internal</code>. For more information, see <u>Restrict network ingress for Cloud Run</u> (/run/docs/securing/ingress). Data encryption: By default, Cloud Run encrypts data by using a Google-owned and Google-managed encryption key. To protect your containers by using a key that you control, you can use customer-managed encryption keys (CMEK). For more information, see <u>Using customer managed encryption keys</u> (/run/docs/securing/using-cmek). Container image security: To ensure that only authorized container images are deployed to the Cloud Run services, you can use <u>Binary Authorization</u> (/binary-authorization/docs/run/overview). Data residency: Cloud Run helps you to meet data residency requirements. Cloud Run container instances run within the region that you select. For more guidance about container security, see <u>General Cloud Run development tips</u> (/run/docs/tips/general#container-security).
Cloud Storage	Data encryption: By default, the data that's stored in Cloud Storage is encrypted using Google-owned and Google-managed encryption keys. If required, you can use CMEKs or your own keys that you manage by using an external management method like customer-supplied encryption keys (CSEKs). For more information, see <u>Data encryption options</u> (/storage/docs/encryption).

Product	Design considerations and recommendations
	Access control: Cloud Storage supports two methods for controlling user access to your buckets and objects: Identity and Access Management (IAM) and access control lists (ACLs). In most cases, we recommend using IAM, which lets you grant permissions at the bucket and project levels. For more information, see Overview of access control (/storage/docs/access-control). Data protection: The data that you load into the data ingestion subsystem through Cloud Storage might include sensitive data. To protect such data, you can use Sensitive Data Protection to discover, classify, and de-identify the data. For more information, see Using Sensitive Data Protection with Cloud Storage (/sensitive-data-protection/docs/dlp-gcs). Network control: To mitigate the risk of data exfiltration from Cloud Storage, you can create a service perimeter by using VPC Service Controls (/vpc-service-controls/docs/supported-products#table_storage). Data residency: Cloud Storage helps you to meet data residency requirements. Data is stored or replicated within the regions that you specify.
Pub/Sub	Data encryption: By default, Pub/Sub encrypts all messages, both at rest and in transit, by using Google-owned and Google-managed encryption keys. Pub/Sub supports the use of CMEKs for message encryption at the application layer. For more information, see Configure message encryption (/pubsub/docs/encryption). Data residency: If you have data residency requirements, in order to ensure that message data is stored in specific locations, you can configure message storage policies (/pubsub/docs/resource-location-restriction).
Cloud Logging	Administrative activity audit: Logging of administrative activity (/logging/docs/audit#admin-activity) is enabled by default for all of the Google Cloud services that are used in this reference architecture. You can access the logs through Cloud Logging and use the logs to monitor API calls or other actions that modify the configuration or metadata of Google Cloud resources. Data access audit: Logging of data access events (/logging/docs/audit#data-access) is enabled by default for BigQuery. For the other services that are used in this architecture, you can enable Data Access audit logs. You can use these logs to monitor the following: • API calls that read the configuration or metadata of resources. • User requests to create, modify, or read user-provided resource data. Security of log data: Google doesn't access or use the data in Cloud Logging. Data residency: To help meet data residency requirements, you can configure Cloud Logging to store log data in the region that you specify. For more information, see Regionalize your logs (/logging/docs/regionalized-logs).
All of the products in	Mitigate data exfiltration risk: To reduce the risk of data exfiltration, create a VPC Service Controls (/vertex-ai/docs/general/vpc-service-controls) perimeter around the

Product	Design considerations and recommendations
the architecture	infrastructure. VPC Service Controls supports all of the services that are used in this reference architecture. Post-deployment optimization: After you deploy your application in Google Cloud, use the Active Assist (/recommender/docs/whatis-activeassist) service to get recommendations that can help you to further optimize the security of your cloud resources. Review the recommendations and apply them as appropriate for your environment. For more information, see Find recommendations in Active Assist (/recommender/docs/recommendation-hub/find-recommendations). Access control: Follow the principle of least privilege (/iam/docs/using-iam-securely#least_privilege) for every cloud service.

For general guidance regarding security for AI and ML deployments in Google Cloud, see the following resources:

- (Blog) [Introducing Google's Secure AI Framework](https://blog.google/technology/safety-security/introducing-googles-secure-ai-framework/)
(<https://blog.google/technology/safety-security/introducing-googles-secure-ai-framework/>)
- (Documentation) [AI and ML security perspective](/architecture/framework/perspectives/ai-ml/security)
(</architecture/framework/perspectives/ai-ml/security>) in the Google Cloud Well-Architected Framework
- (Documentation) [Vertex AI shared responsibility](/vertex-ai/docs/shared-responsibility) (</vertex-ai/docs/shared-responsibility>)
- (Whitepaper) [Generative AI, Privacy, and Google Cloud](https://services.google.com/fh/files/misc/genai_privacy_google_cloud_202308.pdf)
(https://services.google.com/fh/files/misc/genai_privacy_google_cloud_202308.pdf)
- (Video) [Protecting sensitive data in AI apps](https://www.youtube.com/watch?v=ct1nuLS-wIA)
(<https://www.youtube.com/watch?v=ct1nuLS-wIA>)

Reliability

This section describes design considerations and recommendations to build and operate reliable infrastructure for your deployment in Google Cloud.

Product	Design considerations and recommendations
Vector Search	Query scaling: To make sure that the Vector Search index can handle increases in query load, you can configure autoscaling for the index endpoint. When the query load increases, the number of nodes is increased automatically up to the maximum that you specify. For more information, see Enable autoscaling (/vertex-ai/docs/vector-search/deploy-index-vpc#autoscaling).

Product	Design considerations and recommendations
Cloud Run	Robustness to infrastructure outages: Cloud Run is a regional service. Data is stored synchronously across multiple zones within a region. Traffic is automatically load-balanced across the zones. If a zone outage occurs, Cloud Run continues to run and data isn't lost. If a region outage occurs, Cloud Run stops running until Google resolves the outage.
Cloud Storage	Data availability: You can create Cloud Storage buckets in one of three location types (/storage/docs/locations): regional, dual-region, or multi-region. Data that's stored in regional buckets is replicated synchronously across multiple zones within a region. For higher availability, you can use dual-region or multi-region buckets, where data is replicated asynchronously across regions.
Pub/Sub	Rate control: To avoid errors during periods of transient spikes in message traffic, you can limit the rate of publish requests by configuring flow control (/pubsub/docs/flow-control-messages) in the publisher settings. Failure handling: To handle failed publish attempts, adjust the retry-request variables as necessary. For more information, see Retry requests (/pubsub/docs/retry-requests).
BigQuery	Robustness to infrastructure outages: Data that you load into BigQuery is stored synchronously in two zones within the region that you specify. This redundancy helps to ensure that your data isn't lost when a zone outage occurs. For more information about reliability features in BigQuery, see Understand reliability (/bigquery/docs/reliability-intro).
All of the products in the architecture	Post-deployment optimization: After you deploy your application in Google Cloud, use the Active Assist (/recommender/docs/whatis-activeassist) service to get recommendations to further optimize the reliability of your cloud resources. Review the recommendations and apply them as appropriate for your environment. For more information, see Find recommendations in Active Assist (/recommender/docs/recommendation-hub/find-recommendations).

For reliability principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Reliability](#) (/architecture/framework/perspectives/ai-ml/reliability) in the Well-Architected Framework.

Cost optimization

This section provides guidance to optimize the cost of setting up and operating a Google Cloud topology that you build by using this reference architecture.

Product	Design considerations and recommendations
Vector Search	Billing for Vector Search depends on the size of your index, queries per second (QPS), and the number and machine type of the nodes that you use for the index endpoint. For high-

Product	Design considerations and recommendations
	QPS workloads, batching the queries can help to reduce cost. For information about how you can estimate Vector Search cost, see Vector Search pricing examples (/vertex-ai/pricing#pricing_examples). To improve the utilization of the compute nodes on which the Vector Search index is deployed, you can configure autoscaling for the index endpoint. When demand is low, the number of nodes is reduced automatically to the minimum that you specify. For more information, see Enable autoscaling (/vertex-ai/docs/vector-search/deploy-index-vpc#autoscaling).
Cloud Run	When you create Cloud Run services, you specify the amount of memory and CPU to be allocated to the container instance. To control costs, start with the default (minimum) CPU and memory allocations. To improve performance, you can increase the allocation by configuring the CPU limit and memory limit. For more information, see the following documentation: • Configure memory limits for services (/run/docs/configuring/services/memory-limits) • Configure CPU limits for services (/run/docs/configuring/services/cpu) If you can predict the CPU and memory requirements of your Cloud Run services, then you can save money by getting discounts for committed usage. For more information, see Cloud Run committed use discounts (/run/cud).
Cloud Storage	For the Cloud Storage bucket that you use to load data into the data ingestion subsystem, choose an appropriate storage class (/storage/docs/storage-classes). When you choose the storage class, consider the data-retention and access-frequency requirements of your workloads. For example, to control storage costs, you can choose the Standard class and use Object Lifecycle Management (/storage/docs/lifecycle). Doing so enables automatic downgrade of objects to a lower-cost storage class or deletion of objects based on conditions that you set.
Cloud Logging	To control the cost of storing logs, you can do the following: • Reduce the volume of logs by excluding or filtering unnecessary log entries. For more information, see Exclusion filters (/logging/docs/exclusions). • Reduce the period for which log entries are retained. For more information, see Configure custom retention (/logging/docs/buckets#custom-retention).
BigQuery	BigQuery lets you estimate the cost of queries before you run them. To optimize query costs, you need to optimize storage and query computation. For more information, see Estimate and control costs (/bigquery/docs/best-practices-costs).

Product	Design considerations and recommendations
---------	---

All of the products in the architecture	After you deploy your application in Google Cloud, use the Active Assist (/recommender/docs/whatis-activeassist) service to get recommendations to further optimize the cost of your cloud resources. Review the recommendations and apply them as appropriate for your environment. For more information, see Find recommendations in Active Assist (/recommender/docs/recommendation-hub/find-recommendations) .
---	--

To estimate the cost of your Google Cloud resources, use the [Google Cloud Pricing Calculator](https://cloud.google.com/products/calculator) (<https://cloud.google.com/products/calculator>).

For cost optimization principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Cost optimization \(/architecture/framework/perspectives/ai-ml/cost-optimization\)](/architecture/framework/perspectives/ai-ml/cost-optimization) in the Well-Architected Framework.

Performance optimization

This section describes design considerations and recommendations to design a topology in Google Cloud that meets the performance requirements of your workloads.

Product	Design considerations and recommendations
---------	---

Vector Search	When you create the index, set the shard size, distance measure type, and number of embeddings for each leaf node based on your performance requirements. For example, if your application is extremely sensitive to latency variability, we recommend a large shard size. For more information, see Configuration parameters that affect performance (/vertex-ai/docs/vector-search/create-manage-index#performance-parameters) .
---------------	---

When you configure the compute capacity of the nodes on which the Vector Search index is deployed, consider your requirements for performance. Choose an appropriate machine type and set the maximum number of nodes based on the query load that you expect. For more information, see [Deployment settings that affect performance \(/vertex-ai/docs/vector-search/deploy-index-vpc#performance\)](/vertex-ai/docs/vector-search/deploy-index-vpc#performance).

Configure the query parameters for the Vertex Search index based on your requirements for query performance, availability, and cost. For example, the **approximateNeighborsCount** parameter specifies the number of neighbors that must be retrieved before exact reordering is performed. Decreasing the value of this parameter can help to reduce latency and cost. For more information, see [Query-time settings that affect performance \(/vertex-ai/docs/vector-search/query-index-vpc#performance\)](/vertex-ai/docs/vector-search/query-index-vpc#performance).

An index that's up-to-date helps to improve the accuracy of the generated responses. You can update your Vector Search index by using batch or streaming updates. Streaming

Product	Design considerations and recommendations
	updates let you perform near real-time queries on updated data. For more information, see Update and rebuild an active index (/vertex-ai/docs/vector-search/update-rebuild-index).
Cloud Run	By default, each Cloud Run container instance is allocated one CPU and 512 MiB of memory. Depending on the performance requirements, you can configure the CPU limit and the memory limit. For more information, see the following documentation: • Configure memory limits for services (/run/docs/configuring/services/memory-limits) • Configure CPU limits for services (/run/docs/configuring/services/cpu) To ensure optimal latency even after a period of no traffic, you can configure a minimum number of instances (/run/docs/configuring/min-instances). When such instances are idle, the CPU and memory that are allocated to the instances are billed at a lower price. For more performance optimization guidance, see General Cloud Run development tips (/run/docs/tips/general#optimize_performance).
Cloud Storage	To upload large files, you can use a method called parallel composite uploads. With this strategy, the large file is split into chunks. The chunks are uploaded to Cloud Storage in parallel and then the data is recomposed in the cloud. When network bandwidth and disk speed aren't limiting factors, then parallel composite uploads can be faster than regular upload operations. However, this strategy has some limitations and cost implications. For more information, see Parallel composite uploads (/storage/docs/parallel-composite-uploads).
BigQuery	BigQuery provides a query execution graph that you can use to analyze query performance and get performance insights for issues like slot contention and insufficient shuffle quota. For more information, see Get query performance insights (/bigquery/docs/query-insights). . After you address the issues that you identify through query performance insights, you can further optimize queries by using techniques like reducing the volume of input and output data. For more information, see Optimize query computation (/bigquery/docs/best-practices-performance-compute).
All of the products in the architecture	After you deploy your application in Google Cloud, use the Active Assist (/recommender/docs/whatis-activeassist) service to get recommendations to further optimize the performance of your cloud resources. Review the recommendations and apply them as appropriate for your environment. For more information, see Find recommendations in Active Assist (/recommender/docs/recommendation-hub/find-recommendations).

For performance optimization principles and recommendations that are specific to AI and ML workloads, see [AI and ML perspective: Performance optimization](/architecture/framework/perspectives/ai-ml/performance-optimization) (/architecture/framework/perspectives/ai-ml/performance-optimization) in the Well-Architected Framework.

Deployment

To deploy a topology that's based on this reference architecture, you can download and use the Terraform sample configuration that's available in a repository in [GitHub](https://github.com/GoogleCloudPlatform/architecture-center-samples/tree/main/gen-ai-rag-vertex-ai-vector-search)

(<https://github.com/GoogleCloudPlatform/architecture-center-samples/tree/main/gen-ai-rag-vertex-ai-vector-search>)

. Follow the instructions in the README in the repository. The sample code isn't intended for production use cases.

What's next

- [Choose models and infrastructure for your generative AI application](#)
([/docs/generative-ai/choose-models-infra-for-ai](#))
- [RAG infrastructure for generative AI using Vertex AI and AlloyDB for PostgreSQL](#)
([/architecture/rag-capable-gen-ai-app-using-vertex-ai](#))
- [RAG infrastructure for generative AI using GKE and Cloud SQL](#)
([/architecture/rag-capable-gen-ai-app-using-gke](#))
- [RAG infrastructure for generative AI using Gemini Enterprise and Vertex AI](#)
([/architecture/rag-genai-agentspace-vertexai](#))
- [GraphRAG infrastructure for generative AI using Vertex AI and Spanner Graph](#)
([/architecture/gen-ai-graphrag-spanner](#))
- For an overview of architectural principles and recommendations that are specific to AI and ML workloads in Google Cloud, see the [AI and ML perspective](#)
([/architecture/framework/perspectives/ai-ml](#)) in the Well-Architected Framework.
- For more reference architectures, diagrams, and best practices, explore the [Cloud Architecture Center](#) ([/architecture](#)).

Contributors

Author: [Kumar Dhanagopal](https://www.linkedin.com/in/kumardhanagopal) (<https://www.linkedin.com/in/kumardhanagopal>) | Cross-Product Solution Developer

Other contributors:

- [Assaf Namer](https://www.linkedin.com/in/assaf-namer-0677644) (https://www.linkedin.com/in/assaf-namer-0677644) | Principal Cloud Security Architect
- Deepak Michael | Networking Specialist Customer Engineer
- [Divam Anand](https://www.linkedin.com/in/divam-anand-25469b50) (https://www.linkedin.com/in/divam-anand-25469b50) | Product Strategy and Operations Lead
- [Eran Lewis](https://www.linkedin.com/in/eranlewis) (https://www.linkedin.com/in/eranlewis) | Senior Product Manager
- [Jerome Simms](https://www.linkedin.com/in/jeromesimms) (https://www.linkedin.com/in/jeromesimms) | Director, Product Management
- Katie McLaughlin | Senior Developer Relations Engineer
- [Mark Schlagenhauf](https://www.linkedin.com/in/mark-schlagenhauf-63b98) (https://www.linkedin.com/in/mark-schlagenhauf-63b98) | Technical Writer, Networking
- [Megan O'Keefe](https://www.linkedin.com/in/askmeegs) (https://www.linkedin.com/in/askmeegs) | Developer Advocate
- [Nicholas McNamara](https://www.linkedin.com/in/nsbm) (https://www.linkedin.com/in/nsbm) | Product and Commercialization Strategy Principal
- [Preston Holmes](https://www.linkedin.com/in/ptone) (https://www.linkedin.com/in/ptone) | Outbound Product Manager - App Acceleration
- [Rob Edwards](https://www.linkedin.com/in/robedwards) (https://www.linkedin.com/in/robedwards) | Technology Practice Lead, DevOps
- [Victor Moreno](https://www.linkedin.com/in/vimoreno) (https://www.linkedin.com/in/vimoreno) | Product Manager, Cloud Networking
- [Wietse Venema](https://www.linkedin.com/in/wietse-venema) (https://www.linkedin.com/in/wietse-venema) | Developer Relations Engineer

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-03-07 UTC.